# Compilation Server for Bots

AWS Lambda now allows to configure functions to run on Arm-based AWS Graviton2 processors in addition to x86-based functions. Using this processor architecture option allows you to get up to 34% better price performance.

Functions using the Arm architecture benefit from the performance and security built into the Graviton2 processor, which is designed to deliver up to 19% better performance for compute-intensive workloads.

Below is described how to create, configure and use a dedicated AWS EC2 instance for this purpose.

## Launching EC2 instance to compile bots

In the AWS EC2 Web Console, launch a new instance with the following parameters:

| Parameter | Value | SBOX specific value | PROD specific value |
|---|---|---|---|
| Architecture | 64 bit (arm) | | |
| Instance type | a1.medium | | |
| Key pair name | new or exising pair | ppcdevdocker | ppc-prod-docker |
| VPC ID | The same as for the deviceproc server | PPC-Dev-Public | PPC-Production |
| Security Group | The same as for the deviceproc server | PPC-Dev-Pub-Security | PPC-Production-Pub |
| Subnet ID | A subnet with the Availability Zone: us-east-1a, or us-east-1b, or us-east-1e | PPC-Dev-Public-Lambda01 | PPC-Production-Lambda01 |
| Auto-assign public IP | false | | |

The bot deployment workflow only needs a SSH access to the new EC2 instance *(compilation server)* from the EC2 instance running the deviceproc module *(deviceproc server)*.

## Setting up SSH access from *deviceproc*

After launching the new instance it is automatically assigned a private IPv4 address.

Since the deviceproc server and the compilation server belong to the same VPC, we can use a private IPv4 address to connect to the compilation server.

1. Copy the *.pem* file to `/root/.ssh` catalog on the EC2 instance that runs the deviceproc module.
2. Open a SSH session to the EC2 instance that runs the deviceproc module, and sudo `sudo -i`

3. Change the permissions of the *.pem file* so only the root user can read it:
   `chmod 400 ~/.ssh/<pem-file>`
4. Open a SSH connection to the new EC2 instance (compilation server):
   `ssh -i ~/.ssh/<pem-file> ec2-user@<build-instance-private-ipv4-address>`
   This command automatically adds the new instance's IP address to the `known_hosts` file.

## Installing Docker on the compilation server

Open a SSH connection to the new EC2 instance from the *deviceproc* server as shown in the last item of the previous section. Run the following commands on the new EC2 instance:

1. Apply pending updates using the yum command:
   `$ sudo yum update`
2. Search for Docker package:
   `$ sudo yum search docker`
3. Get version information:
   `$ sudo yum info docker`
4. Install docker, run:
   `$ sudo yum install docker`
5. Add group membership for the default ec2-user so you can run all docker commands without using the sudo command:
   `$ sudo usermod -a -G docker ec2-user`
   `$ id ec2-user`
   `# Reload a Linux user's group assignments to docker w/o logout`
   `$ newgrp docker`
6. Enable docker service at AMI boot time:
   `$ sudo systemctl enable docker.service`
7. Start the Docker service:
   `$ sudo systemctl start docker.service`

## Verification

Check that the docker daemon on the server is running and accessible to the deviceproc module.

Execute the following command from the *deviceproc* server:

```
$ sudo -i
$ ssh -i ~/.ssh/<pem-file> ec2-user@<build-instance-private-ipv4-address> docker image ls
```

The command output should contain the list of local repositories with the header line:

```
REPOSITORY                    TAG                   IMAGE ID       CREATED      SIZE
```

## ECR Credentials Helper

The Amazon ECR Docker Credential Helper is a credential helper for the Docker daemon that makes it easier to use Amazon Elastic Container Registry.

GitHub - awslabs/amazon-ecr-credential-helper: Automatically gets credentials for Amazon ECR on docker push/docker pull

Steps:

1. Install `amazon-ecr-credential-helper`
2. Replace the existing content of `~/.docker/config.json` with the following text:

```
{
    "credsStore": "ecr-login"
```

```
}
```

3. Create a file `~/.aws/credentials` with the following content where dots should be replaced by the actual values:

```
[default]
aws_access_key_id = ...
aws_secret_access_key = ...
```

4. Set access rights for the file:

```
$ chmod 0600 .aws/credentials
```

## Docker builder instances

What we call “compiling a bot” is creating a container image that will run on AWS Lambda. Docker provides a default builder that is responsible for creating the image. But there is a problem with this builder — the “no space left on device” error.

With each bot compilation, the Docker builder cache gets larger. Eventually, this will lead to a situation where there is no space left on the device. Docker does not directly support limiting the cache by time or size, so this can only be done with scheduled cleanups.

Clearing the cache periodically solves the problem of lack of disk space, but can lead to an unpleasant situation - an infinite loop if clearing the cache and compiling the bot accidentally happen at the same time.

A good solution to the problem is to have two different builder instances, one in use and one reserved, switching between them periodically. The cache of one instance can be safely cleared while the other instance is used to compile bots.

This dual-builder architecture can be implemented in Docker using dedicated BuildKit containers. Each container provides an isolated environment for running builds, including a separate cache.

**Configure dual-builder architecture**

1. Install BuildKit container image in Docker:
`$ docker pull moby/buildkit`

2. Create and start 2 builder instances: *bot_builder_1* and *bot_builder_2*:
`$ docker buildx create --driver-opt image=moby/buildkit --bootstrap --name bot_builder_1`
`$ docker buildx create --driver-opt image=moby/buildkit --bootstrap --name bot_builder_2`

3. Create file *bot_builders* with 2 lines containing the builder instance names:

```
bot_builder_1
bot_builder_2
```

The first line contains the name of the builder instance currently in use.

4. Create a shell script `swap_builders.sh` that clears the cache of reserved builder instance and then makes the reserved instance active

```
#!/bin/bash

FILE=/home/ec2-user/bot_builders

# Read the builder instance names from the file
BUILDER_1=$(sed -n '1p' $FILE)  # active
BUILDER_2=$(sed -n '2p' $FILE)  # reserved

# Clear the reserved buider's cache
docker buildx prune --builder "${BUILDER_2}" --force

```

```
# Swap the builder names write them to the temp file
echo "${BUILDER_2}" > ${FILE}.tmp
echo "${BUILDER_1}" >> ${FILE}.tmp

# Replace the original file by the temp file
mv ${FILE}.tmp $FILE
```

5. Make the script executable and create a crontab task for daily cleanup of the build cache

```
@daily /home/ec2-user/swap_builders.sh "switch between builders"
```

6. Create a shell script ‘compile_bot.sh’

```
#!/bin/bash

# Read the name of the builder to use from the first line of the 'bot_builders' file
BUILDER_NAME=$(sed -n '1p' bot_builders)

# Run docker build command
#
# --builder     Use a specific builder instance.
# --provenance  Provenance attestation. Must be disabled for AWS Lambda compatibility, see https://github.com/docker/buildx/issues/1533
# --tag         Image URI, ex. 846566076533.dkr.ecr.us-east-1.amazonaws.com/bot-dev-2:8ead89d4-985c-498f-9621-9616ca88eed1
# --pull        Always attempt to pull the latest versions of all referenced images.
# --push        Automatically push the created image to the remote AWS ECR registry identified by the image URI.
# -             Read dockerfile from stdin.
docker buildx build --builder "$BUILDER_NAME" --provenance=false --pull --push --tag "$1" -
```

The script should be referenced in the main server’s system property “ppc.bot.build.command”. For the Sbox cloud it is:

| | |
|---|---|
| ppc.bot.build.command | ssh -q -o StrictHostKeyChecking=no -i ~/.ssh/ppcdevdocker.pem ec2-user@172.19.3.84 /home/ec2-user/compile_bot.sh |

## Links

1. AWS Graviton Processor
2. Migrating AWS Lambda functions to Arm-based AWS Graviton2 processors
3. How to install Docker on Amazon Linux 2
4. Amazon EC2 On-Demand Pricing
5. Amazon EC2 Reserved Instances Pricing
6. AWS EC2: Converting from On Demand to Reserved Instance